Appunti del Laboratorio di Ottica

Mirolo Manuele
Alessio Brusini
Michele Tedeschini
Emanuele Stroili

Anno Accademico 2026/2027

Indice

1 Introduzione all'Ottica **1**

1.1 Esempi di Formattazione 1

1.2 Info 1

1.3 Onde elettromagnetiche e approssimazione Iconale 10

1.3.1 Principio di Fermat e Cammino Ottico 11

1.4 Esperienza 1: Verifica dell'Irradianza di una sorgente puntiforme 13

1.4.1 Modello Teorico 13

1.4.2 Set-up Sperimentale e Procedura 14

1.4.3 Analisi Dati e Preparazione per ROOT 15

1.4.4 Grafici Diagnostici Finali 16

1

Introduzione all'Ottica

1.1 Esempi di Formattazione

Un esperimento tipico richiede la misura della lunghezza d'onda di un laser He-Ne. Secondo i nostri dati, il valore misurato è $\lambda = (632{,}8 \pm 0{,}1)\,\text{nm}$.

$$f(x) = \sin\left(\frac{2\pi}{\lambda}x\right) \tag{1.1}$$

Definizione L'ottica geometrica è l'approssimazione in cui la lunghezza d'onda della luce è considerata molto minore delle dimensioni degli oggetti con cui interagisce ($\lambda \to 0$).

Nota *Ricordarsi di allineare sempre gli specchi dell'interferometro di Michelson prima di iniziare la presa dati*

Teorema : Legge di Snell

Dati due mezzi con indici di rifrazione n_1 e n_2, l'angolo di incidenza θ_1 e l'angolo di rifrazione θ_2 sono legati da:

$$n_1 \sin\theta_1 = n_2 \sin\theta_2 \tag{1.2}$$

1.2 Info

8 esperienze relazioni d'esame entro 1 settimana prima dell'appello (le prime relazioni andranno spedite entro 1 settimana) max gruppo da 5 persone (5 gruppi per giorno). 1 solo set di relazioni per gruppo

Introduzione: Analisi Dati con Python (Facoltativo)

In questo capitolo vengono introdotti gli strumenti software per l'analisi statistica dei dati sperimentali. Il framework di riferimento è **Python**, in particolare utilizzando la libreria `iminuit` per l'ottimizzazione dei parametri tramite il metodo dei minimi quadrati (χ^2).

Fit di un Istogramma con una Gaussiana

Il primo esempio mostra come generare un set di dati distribuiti secondo una normale (Gaussiana), riempirne un istogramma e successivamente effettuare un fit per estrarre i parametri della distribuzione.

La funzione Gaussiana utilizzata per il fit è definita come:

$$f(x; C, \mu, \sigma) = C \exp\left(-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right) \tag{1.3}$$

Di seguito il codice completo:

Listing 1.1: Generazione e fit di un istogramma Gaussiano

```
import numpy as np
import matplotlib.pyplot as plt
from iminuit import Minuit

# 1. Generazione dati fittizi (Gaussiana con mu=-5.5, sigma=0.5)
with open('datpy.txt','w') as file:
  for i in range(1000):
    g = np.random.normal(-5.5, 0.5)
    file.write('%s\n' % (g))

# 2. Lettura dati e creazione istogramma
dati = np.loadtxt("datpy.txt")
x = dati

plt.figure()
counts, bins, _ = plt.hist(x, range=(-10,0), bins=50, alpha=0.6)

# 3. Definizione del modello e del Chi Quadrato
def gauss(x, par0, par1, par2):
    return par0 * np.exp(-0.5 * np.power((x - par1) / par2, 2.0))

def chi2(par0, par1, par2):
    mask = counts > 0 # Evita la divisione per zero nei bin vuoti
    # x dei bin valutato nel punto medio
    x_mid = (bins[:-1] + bins[1:])[mask] / 2
    y_teo = gauss(x_mid, par0, par1, par2)
    err_y = np.sqrt(counts[mask])
    return np.sum(((counts[mask] - y_teo) / err_y)**2)

```

```
# 4. Minimizzazione con Minuit
m = Minuit(chi2, par0=1.0, par1=0.0, par2=1.0)
m.migrad() # Algoritmo di minimizzazione
m.hesse()  # Calcolo della matrice di covarianza
m.minos()  # Stima asimmetrica degli errori (opzionale)

# Estrazione dei parametri
Constant_fit = m.values["par0"]
Mean_fit = m.values["par1"]
Sigma_fit = m.values["par2"]

# [...] (Codice per il plot omesso per brevità)
plt.show()
```

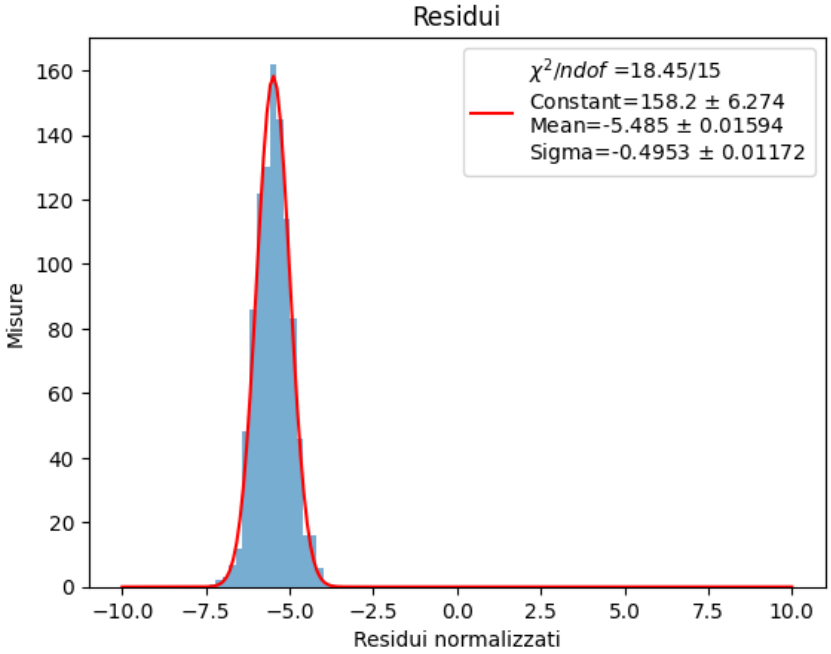


Figura 1.1: Fit di un set di dati generati casualmente secondo una distribuzione Normale ($\mu = -5.5, \sigma = 0.5$). La curva rossa rappresenta la funzione di fit ottimizzata tramite minimizzazione del χ^2. Il box in alto a destra, generato da `iminuit`, riporta i parametri estratti e i relativi errori.

Fit non lineare, errori su ambo gli assi e Analisi dei Residui

Questo secondo script simula un decadimento esponenziale $y = Ae^{-x/B}$ ed esegue un'analisi completa in tre fasi:

1. **Fit del modello:** Esegue la minimizzazione considerando la varianza efficace.
2. **Distribuzione dei residui:** Verifica che i residui normalizzati, definiti come $r_i = \frac{y_i - f(x_i)}{\sigma_{eff,i}}$, seguano una distribuzione Normale standard $N(0, 1)$.

3. **Andamento dei residui:** Esegue un fit lineare ($y = P_0 + P_1 x$) sul grafico dei residui in funzione di x. Se il modello iniziale è corretto, non ci dovrebbero essere trend sistematici (ci si aspetta $P_0 \simeq 0, P_1 \simeq 0$).

Listing 1.2: Fit esponenziale con varianza efficace e studio dei residui

```
import numpy as np
import matplotlib.pyplot as plt
from iminuit import Minuit

# --- 1. LETTURA DATI ---
dati = np.loadtxt("datpy.txt")
x = dati[:,0]
y = dati[:,1]
ex = dati[:,2]
ey = dati[:,3]

# --- 2. FIT ESPONENZIALE (METODO VARIANZA EFFICACE) ---
def fitfunc(x, par0, par1):
    return par0 * np.exp(-x / par1)

def fitfuncprime(x, par0, par1):
    dx = 1e-6
    # Derivata numerica
    return (fitfunc(x+dx/2, par0, par1) - fitfunc(x-dx/2, par0, par1))/dx

def chi2_eff(par0, par1):
    # Propagazione dell'errore di x su y tramite derivata
    sigma_eff = np.sqrt(ey**2 + (ex * fitfuncprime(x, par0, par1))**2)
    return np.sum(((y - fitfunc(x, par0, par1)) / sigma_eff)**2)

m1 = Minuit(chi2_eff, par0=1.0, par1=1.0)
m1.migrad()
A_fit, B_fit = m1.values["par0"], m1.values["par1"]

# --- 3. ANALISI DISTRIBUZIONE DEI RESIDUI ---
y_res = y - fitfunc(x, A_fit, B_fit)
ey_eff = np.sqrt(ey**2 + (ex * fitfuncprime(x, A_fit, B_fit))**2)

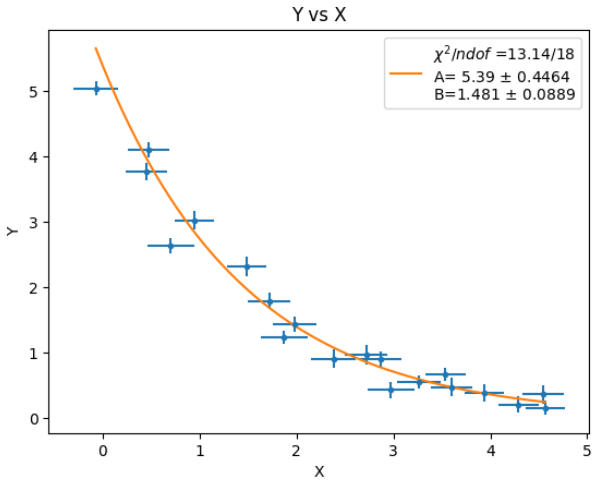
counts, bins, _ = plt.hist(y_res / ey_eff, range=(-10,10), bins=20)

def gauss(x, par0, par1, par2):
    return par0 * np.exp(-0.5 * np.power((x - par1) / par2, 2.0))

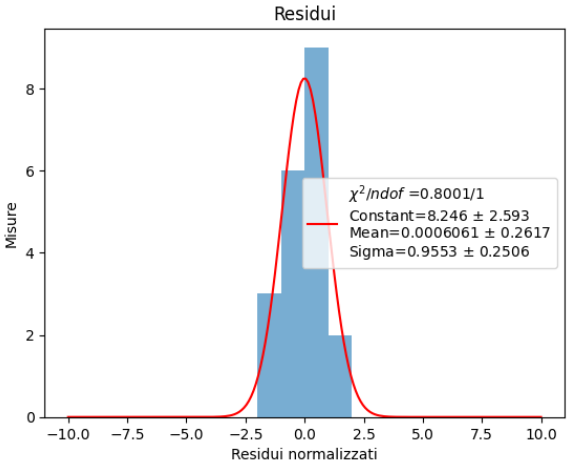
def chi2_res(par0, par1, par2):
    mask = counts>0
    x_mid = (bins[:-1] + bins[1:])[mask]/2
    return np.sum(((counts[mask] - gauss(x_mid, par0, par1, par2)) / np.sqrt(
    counts[mask]))**2)

m2 = Minuit(chi2_res, par0=1.0, par1=0.0, par2=1.0)
m2.migrad()
```

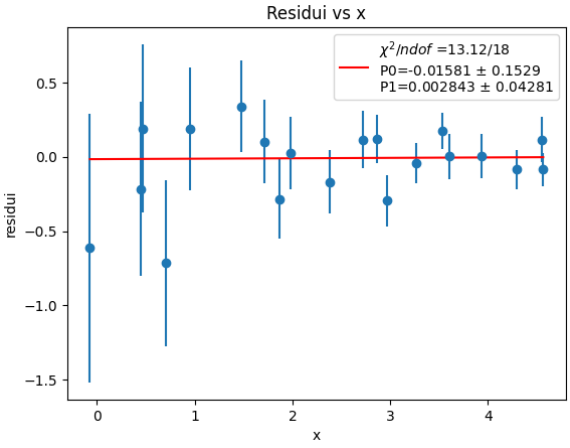
```
46
47 # --- 4. RICERCA TREND SISTEMATICI SUI RESIDUI ---
48 def retta(x, par0, par1):
49     return par0 + par1 * x
50
51 def chi2_trend(par0, par1):
52     return np.sum(((y_res - retta(x, par0, par1)) / ey_eff)**2)
53
54 m3 = Minuit(chi2_trend, par0=0.0, par1=0.0)
55 m3.migrad()
```



(a) Fit del modello esponenziale. Gli errori sulle y includono la varianza efficace derivata dall'incertezza sulle x.



(b) Distribuzione dei residui normalizzati. Il fit Gaussiano conferma che $r_i \sim N(0,1)$, indicando una corretta stima degli errori.



(c) Andamento dei residui in funzione di x. Il fit lineare non mostra pendenze significative ($P_1 \simeq 0$), escludendo errori sistematici di modello.

Figura 1.2: Analisi completa del fit non lineare ($y = Ae^{-x/B}$) con l'utilizzo della varianza efficace e studio dei residui.

Il Framework ROOT

ROOT può essere utilizzato in due modi, entrambi illustrati di seguito:

- Tramite **Macro C++** (file `.C`), che vengono compilate o interpretate nativamente.
- Tramite **PyROOT** (file `.py`), un *wrapper* che permette di usare le classi C++ di ROOT all'interno di uno script Python.

Oggetti fondamentali in ROOT Nella sintassi di ROOT, ogni oggetto inizia tipicamente per `T`:

- `TCanvas`: Dove vengono disegnati i grafici.
- `TH1F`: Istogramma monodimensionale (1D) con frequenze in formato *float*.
- `TGraphErrors`: Grafico a dispersione per coppie di dati (x, y) con rispettivi errori (σ_x, σ_y).
- `TF1`: Funzione matematica 1D, utilizzata per eseguire i fit.

Fit Base: Istogramma e Gaussiana (Macro C++)

Questo esempio mostra come leggere una colonna di dati, riempire un istogramma `TH1F` e fittarlo con una funzione di Gauss. In ROOT, funzioni comuni come la Gaussiana sono già predefinite nella libreria matematica `TMath`. Di seguito è riportato il codice in C++ (`fit.C`):

Listing 1.3: Macro C++ per fit Gaussiano con ROOT

```
#include "TMath.h"
#include "TF1.h"
#include "TH1F.h"
#include "TCanvas.h"
#include <fstream>

// Definizione della funzione di fit
double fitfunc(double* x, double* p) {
   double norm = p[0];
   double mean = p[1];
   double sigma = p[2];
   return norm * TMath::Gaus(x[0], mean, sigma);
}

void fit() {
   const int mpt = 9999;
   float x1[mpt];
   int npt = 0;

   // 1. Lettura dati da file
```

```
   std::ifstream in("dat.txt");
   while (in.good()) {
      in >> x1[npt];
      if (!in.good()) break;
      npt++;
   }
   in.close();

   // 2. Creazione Canvas e Istogramma
   TCanvas* c1 = new TCanvas("c1", "c1", 0, 0, 500, 500);
   TH1F* h1 = new TH1F("h1", "h1", 100, -10.0, 25.0);

   for (int i = 0; i < npt; i++) {
     h1->Fill(x1[i]);
   }

   // 3. Definizione TF1 e Fit
   TF1* f1 = new TF1("f1", fitfunc, -100.00, 100.0, 3);
   f1->SetParameters(7.100, -5.100, 0.100);
   f1->SetParNames("A", "B", "C");

   gStyle->SetOptStat(kFALSE);
   gStyle->SetOptFit(kTRUE); // Mostra il box con i risultati del fit

   c1->cd();
   h1->Fit("f1");
   f1->SetLineColor(kRed);
   f1->Draw("same"); // Disegna la funzione sopra l'istogramma
}
```

Fit Avanzato: TGraphErrors e Residui (PyROOT)

Nel caso di misure (x, y) con incertezze su entrambi gli assi, si utilizza la classe `TGraphErrors`. Questo script mostra l'implementazione in **PyROOT**.

Lo script esegue il fit dei dati, calcola gli errori efficaci, costruisce l'istogramma dei residui e verifica l'assenza di trend sistematici tramite un fit lineare (polinomio di grado 1: `pol1`):

Listing 1.4: Fit con Varianza Efficace e Residui tramite PyROOT

```
import ROOT
import array

# 1. Definizione della funzione (Esponenziale)
def fitfunc(x, p):
  return p[0] * ROOT.TMath.Exp(-x[0] / p[1])

# Inizializzazione array tipizzati (necessari per interfacciarsi col C++ di
    ROOT)
```

```
x1, x2, x3, x4 = array.array('f'), array.array('f'), array.array('f'), array.
    array('f')

# [ ... Lettura file datpy.txt omessa per brevità ... ]

npt = len(x1)

# 2. Creazione Graph e Fit Principale
c1 = ROOT.TCanvas('c1', 'Fit Esponenziale', 0, 0, 500, 500)
gr = ROOT.TGraphErrors(npt, x1, x2, x3, x4)

f1 = ROOT.TF1('f1', fitfunc, 0.00, 10.00, 2)
f1.SetParameters(0.0, 1.0)
f1.SetParNames('A', 'B')

ROOT.gStyle.SetOptFit(ROOT.kTRUE)

c1.cd()
gr.Fit(f1) # ROOT si occupa della minimizzazione
gr.SetLineColor(ROOT.kBlue)
gr.Draw('AP') # A=Assi, P=Punti
f1.SetLineColor(ROOT.kRed)
f1.Draw('same')

# 3. Analisi dei Residui Normalizzati (con errore efficace calcolato da ROOT)
x4b = array.array('f') # conterra' gli errori efficaci calcolati

for i in range(npt):
  # Varianza efficace usando f1.Derivative()
  var_eff = x4[i]**2 + (x3[i]*f1.Derivative(x1[i]))**2
  x4b.append(ROOT.TMath.Sqrt(var_eff))

h1 = ROOT.TH1F('h1', 'Distribuzione Residui', 20, -10.0, 10.0)

for i in range(npt):
  resid = (x2[i] - f1.Eval(x1[i])) / x4b[i]
  h1.Fill(resid)

# Fit dei residui con Gaussiana
c2 = ROOT.TCanvas('c2', 'Residui', 600, 0, 500, 500)
fgaus = ROOT.TF1('fgaus', 'gaus', -2.0, 2.0)
fgaus.SetParameters(3.0, 0.0, 1.0)
c2.cd()
h1.Fit(fgaus)

# 4. Trend dei residui vs x
y2 = array.array('f')
for i in range(npt):
  y2.append(x2[i] - f1.Eval(x1[i]))

c3 = ROOT.TCanvas('c3', 'Trend Residui', 300, 300, 500, 500)
```

```
gr1 = ROOT.TGraphErrors(npt, x1, y2, 0, x4b) # X ha errore 0 qui per il plot
g1 = ROOT.TF1('g1', 'pol1', 0.00, 10.00) # Fit lineare

c3.cd()
gr1.Fit(g1)
gr1.Draw('AP')
g1.Draw('same')

# Salvataggio su file
c1.SaveAs('c1.png')
c2.SaveAs('c2.png')
c3.SaveAs('c3.png')
```

Richiamo sulle equazioni di Maxwell

In **presenza di sorgenti**, le equazioni di Maxwell assumono la forma:

$$\nabla \cdot \vec{E}(\vec{r},t) = \frac{\rho}{\varepsilon} \qquad \nabla \times \vec{E}(\vec{r},t) = -\partial_t \vec{B}(\vec{r},t) \tag{1.4}$$

$$\nabla \cdot \vec{B}(\vec{r},t) = 0 \qquad \nabla \times \vec{B}(\vec{r},t) = \mu \left\{ \vec{J}(\vec{r},t) + \varepsilon \partial_t \vec{E}(\vec{r},t) \right\} \tag{1.5}$$

Inoltre sono importanti da ricordare le seguenti relazioni:

$$\nabla \cdot \vec{J}(\vec{r},t) = -\partial_t \rho \quad \text{(Equazione di continuità)} \tag{1.6}$$

$$\vec{F} = q(\vec{E} + \vec{v} \times \vec{B}) \quad \text{(Forza di Lorentz)} \tag{1.7}$$

In presenza di un materiale dielettrico:

$$\vec{D} = \varepsilon_0 \varepsilon_r \vec{E} \qquad \vec{B} = \mu_0 \mu_r \vec{H} \tag{1.8}$$

Se invece siamo in **assenza di sorgenti** ($\rho = 0,\ \vec{J} = 0$), le equazioni diventano:

$$\nabla \cdot \vec{E}(\vec{r},t) = 0 \qquad \nabla \times \vec{E}(\vec{r},t) = -\partial_t \vec{B} \tag{1.9}$$

$$\nabla \cdot \vec{B}(\vec{r},t) = 0 \qquad \nabla \times \vec{B}(\vec{r},t) = \mu\varepsilon \partial_t \vec{E} \tag{1.10}$$

Possiamo ricavare l'equazione delle onde applicando il rotore alla terza equazione:

$$\nabla \times (\nabla \times \vec{E}) = -\nabla^2 \vec{E} + \nabla(\cancelto{0}{\nabla \cdot \vec{E}}) = -\partial_t (\nabla \times \vec{B}) \tag{1.11}$$

Sostituendo il rotore di $\vec{B}$, otteniamo:

$$\nabla^2 \vec{E} = \varepsilon_0 \mu_0 \partial_t^2 \vec{E} \tag{1.12}$$

Eseguendo lo stesso procedimento per il campo magnetico si ottiene un risultato analogo:

$$\begin{cases} \nabla^2 \vec{E} = \varepsilon_0 \mu_0 \partial_t^2 \vec{E} \\ \nabla^2 \vec{B} = \varepsilon_0 \mu_0 \partial_t^2 \vec{B} \end{cases} \tag{1.13}$$

1.3 Onde elettromagnetiche e approssimazione Iconale

La soluzione all'equazione d'onda per campi monocromatici è data da:

$$\vec{E}(\vec{r},t) = \vec{E}_0 e^{i(\vec{k}\cdot\vec{r}-\omega t)} \tag{1.14}$$

$$\vec{B}(\vec{r},t) = \frac{\vec{E}_0}{c} e^{i(\vec{k}\cdot\vec{r}-\omega t)} \tag{1.15}$$

dove $\vec{k}$ è il vettore d'onda. Dalle equazioni di Maxwell è possibile ricavare inoltre le seguenti relazioni di ortogonalità:

$$\vec{k}\cdot\vec{E}_0 = 0, \qquad \vec{k}\cdot\vec{B}_0 = 0, \qquad \vec{k}\times\vec{E}_0 = \omega\vec{B}_0 \tag{1.16}$$

Ottengo dunque 6 equazioni scalari, ciascuna della forma generale:

$$\nabla^2 v(\vec{r},t) = \frac{1}{c^2}\partial_t^2 v(\vec{r},t) \tag{1.17}$$

la cui soluzione generale (tramite sovrapposizione) è:

$$v(\vec{r},t) = \int A(\omega) e^{i\omega t} v_0 e^{i\vec{k}\cdot\vec{r}} d\omega \tag{1.18}$$

In questo modo la soluzione generale può essere espressa come un'onda con ampiezza dipendente dalla posizione e fase dipendente dalla posizione:

$$v(\vec{r},t) = A(\vec{r}) e^{-ik_0 S(\vec{r})} \tag{1.19}$$

dove k_0 e $k_0 S(\vec{r})$ = costante (superfici d'onda). Se supponiamo che l'ampiezza vari lentamente, possiamo sviluppare la funzione iconale S in serie di Taylor nell'intorno di un punto $\vec{r}_0$:

$$A(\vec{r}) \approx A(\vec{r}_0) \tag{1.20}$$

$$S(\vec{r},t) \approx S(\vec{r}_0) + \nabla S(\vec{r}_0)\cdot(\vec{r}-\vec{r}_0) \tag{1.21}$$

Sviluppo in serie (Funzione iconale per approssimare le superfici d'onda come piani ortogonali alla direzione di propagazione):

$$v(\vec{r}) \simeq A(\vec{r}_0)\exp -ik_0 S(\vec{r}_0) - ik_0\nabla S(\vec{r}_0)\cdot(\vec{r}-\vec{r}_0) \tag{1.22}$$

Definisco il vettore d'onda locale e l'indice di rifrazione $n(\vec{r})$:

$$\vec{k}(\vec{r}) = k_0\nabla S(\vec{r}) \quad e \quad n(\vec{r}) = |\nabla S(\vec{r})| \tag{1.23}$$

Da cui il modulo del vettore d'onda: $k(\vec{r}) = |\vec{k}(\vec{r})| = k_0|\nabla S(\vec{r})| = k_0 n(\vec{r})$.

1.3.1 Principio di Fermat e Cammino Ottico

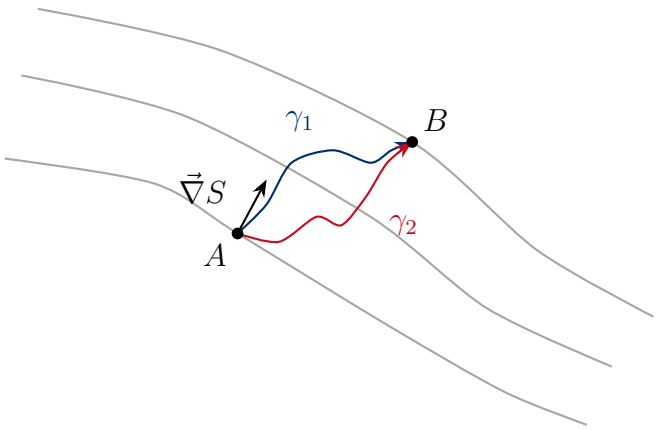


Figura 1.3: Propagazione tra superfici d'onda e percorsi alternativi.

Definisco *cammino ottico* l la seguente grandezza:

$$l = \int_\gamma |\nabla S(\vec{r})| ds \tag{1.24}$$

Vale la seguente disuguaglianza: il percorso più breve è quello a pendenza massima (ortogonale alle superfici d'onda), ovvero lungo le linee di flusso del gradiente:

$$\int_\gamma |\nabla S(\vec{r})| ds \leq \int_{\gamma'} |\nabla S(\vec{r})| ds \tag{1.25}$$

Principio di Fermat Definita la variazione di una funzione δl, il Principio di Fermat afferma che la luce segue un percorso per il quale il cammino ottico è stazionario:

$$\delta l = \delta \int_\gamma |n(\vec{r})| ds = 0 \tag{1.26}$$

Trasporto di energia

Consideriamo un volume V contenente delle cariche in movimento.

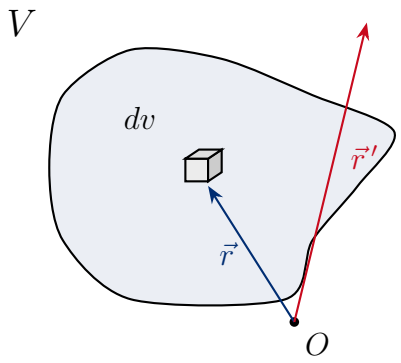


Figura 1.4: Elemento di volume dv all'interno del volume V.

Siano:

- u: densità di energia elettromagnetica.
- $\vec{S}$: densità di flusso di energia (Vettore di Poynting).

Posso definire la variazione di energia elettromagnetica interna al volume come:

$$\Delta E = \partial_t \left\{ \int_V u \, dV \right\} \Delta t \tag{1.27}$$

In presenza di un lavoro esterno, la variazione di energia è data da:

$$\Delta E = \partial_t \left\{ \oint_{\partial V} \vec{S} \cdot \hat{n} \, dA \right\} \Delta t + \left\{ \int_V \frac{\Delta L}{\Delta t} dV \right\} \Delta t \tag{1.28}$$

dove il termine di lavoro meccanico sulle cariche è dato dalla forza di Lorentz:

$$\frac{\Delta L}{\Delta t} = \vec{F} \cdot \vec{v} = \vec{E} \cdot \vec{J}(\vec{r}, t) \tag{1.29}$$

La variazione di energia elettromagnetica nel volume, unita al bilancio dei flussi e del lavoro sulle cariche, ci dà:

$$-\partial_t \int_V u \, dV = \int_A \vec{S} \cdot \hat{n} \, dA + \int_V \vec{E} \cdot \vec{J} \, dV \tag{1.30}$$

Applicando il teorema della divergenza al primo termine sul lato destro:

$$-\partial_t \int_V u \, dV = \int_V \nabla \cdot \vec{S} \, dV + \int_V \vec{E} \cdot \vec{J} \, dV \quad \Longrightarrow \quad -\partial_t u - \nabla \cdot \vec{S} = \vec{E} \cdot \vec{J} \tag{1.31}$$

Riscriviamo il termine $\vec{E} \cdot \vec{J}$ utilizzando la 4ª equazione di Maxwell:

$$\vec{E} \cdot \vec{J} = \vec{E} \cdot \left(\frac{\nabla \times \vec{B}}{\mu_0} - \varepsilon_0 \partial_t \vec{E} \right) = \frac{\vec{E} \cdot (\nabla \times \vec{B})}{\mu_0} - \frac{\varepsilon_0}{2} \partial_t E^2 \tag{1.32}$$

Sfruttando l'identità vettoriale $\nabla \cdot (\vec{E} \times \vec{B}) = \vec{B} \cdot (\nabla \times \vec{E}) - \vec{E} \cdot (\nabla \times \vec{B})$, otteniamo:

$$\vec{E} \cdot \vec{J} = -\frac{\nabla \cdot (\vec{E} \times \vec{B})}{\mu_0} + \frac{\vec{B} \cdot (\nabla \times \vec{E})}{\mu_0} - \frac{\varepsilon_0}{2} \partial_t E^2 \tag{1.33}$$

Sostituendo $\nabla \times \vec{E} = -\partial_t \vec{B}$ dal 3° postulato di Maxwell:

$$\vec{E} \cdot \vec{J} = -\frac{\nabla \cdot (\vec{E} \times \vec{B})}{\mu_0} - \frac{1}{2\mu_0}\partial_t B^2 - \frac{\varepsilon_0}{2}\partial_t E^2 \tag{1.34}$$

Definendo il **Vettore di Poynting** come $\vec{S} = \frac{1}{\mu_0}(\vec{E} \times \vec{B})$ e la densità di energia u_{em}, la relazione diventa il teorema di Poynting:

$$\vec{E} \cdot \vec{J} = -\nabla \cdot \vec{S} - \partial_t u_{em} \tag{1.35}$$

Definiamo infine l'**Irradianza** di un'onda come la media temporale del modulo del vettore di Poynting:

$$I = \langle |\vec{S}| \rangle = \frac{1}{\mu_0}\langle |\vec{E} \times \vec{B}| \rangle = \frac{1}{\mu_0 c}\langle |E|^2 \rangle \tag{1.36}$$

Se considero un'onda piana monocromatica:

$$I = \frac{\varepsilon_0 c}{2}|E|^2 \tag{1.37}$$

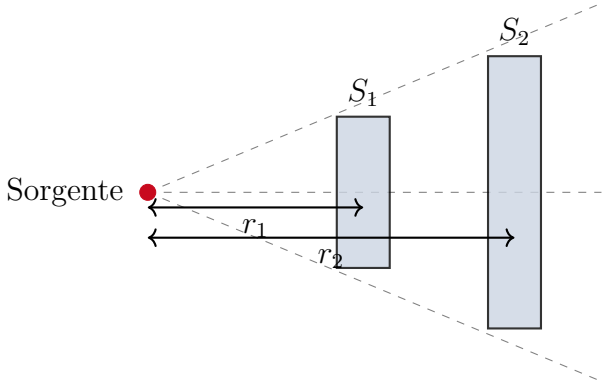
1.4 Esperienza 1: Verifica dell'Irradianza di una sorgente puntiforme

Obiettivo: Realizzare un apparato assimilabile ad una sorgente puntiforme per verificare la legge dell'irradianza per tale sorgente.

1.4.1 Modello Teorico

Consideriamo l'energia che attraversa due porzioni di superfici sferiche S_1 e S_2 a distanze r_1 e r_2. Per la conservazione dell'energia nell'intervallo Δt:

$$I_1 S_1 \Delta t = I_2 S_2 \Delta t \quad \Longrightarrow \quad \frac{I_1}{I_2} = \frac{S_2}{S_1} \tag{1.38}$$

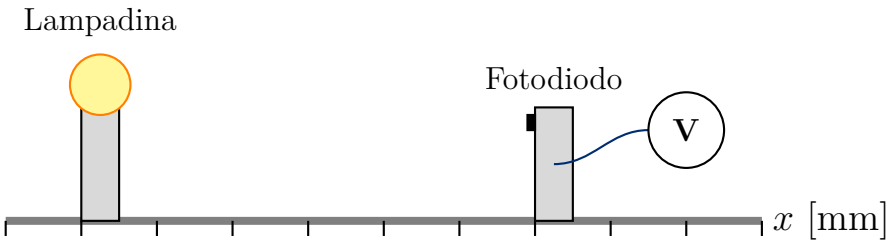


Siccome gli angoli solidi sono uguali, vale la relazione $S_i = \Delta\Omega \cdot r_i^2$. Otteniamo quindi:

$$\frac{I_1}{I_2} = \frac{r_1^2}{r_2^2} \quad \longrightarrow \quad I(r) = I_1 \left(\frac{r_1}{r}\right)^2 \tag{1.39}$$

1.4.2 Set-up Sperimentale e Procedura

L'apparato per la misura prevede un binario graduato (circa 2000 mm) su cui allineare un fotodiodo (con circuito amplificatore montato su supporto mobile) e una lampadina sferica (diametro $D \simeq 40\,\text{mm}$ misurabile con calibro). Il fotodiodo è collegato ad un voltmetro. L'andamento atteso per la tensione letta è $V \sim 1/r^2$.



Per misurare la distanza effettiva r, si pone inizialmente a contatto il fotodiodo con la lampadina alla posizione x_0 (dove x_0 è la posizione iniziale del fotodiodo). La distanza reale per un generico punto x del binario sarà corretta per il diametro della lampadina D e la posizione del sensore (stimata a 1 mm di profondità):

$$r_i = (x_i - x_0) + \frac{D}{2} + 1\,\text{mm} \tag{1.40}$$

(È fondamentale in fase di allineamento che il filamento e la finestra del fotodiodo siano esattamente alla stessa quota per evitare dislivelli, ovvero $h = 0$).

L'errore associato a tale misura spaziale non è puramente casuale. Possiamo stimare l'errore sistematico di traslazione come $(\sigma_{x_0} + \sigma_{D/2} + \sigma_{1\text{mm}})$. Ignoriamo in prima approssimazione per il fit questi ultimi errori, poiché causano solo uno "shift" rigido della scala. L'errore casuale dominante σ_x è dato dalla larghezza della tacca sulla scala graduata (allineamento visivo dell'indice col carrello). Possiamo stimare la deviazione standard tramite una distribuzione uniforme (es. larghezza indice 0,2 mm):

$$\sigma_x \simeq \frac{0{,}2}{\sqrt{3}}\ \text{mm} \tag{1.41}$$

L'incertezza sulle ordinate (Voltaggio) è data invece dalla somma quadratica dell'errore di lettura del multimetro e dell'incertezza sulla fluttuazione di fondo: $\sigma_V = \sqrt{\sigma_{lett}^2 + \sigma_{fondo}^2}$.

Nota

- ***Saturazione e Mascheratura:*** *Si applica del nastro adesivo nero sul fotodiodo lasciando solo una piccola fessura centrale per ridurne l'area attiva. Bisogna regolare la tensione dell'alimentatore per trovare il giusto compromesso affinché il voltmetro non vada a fondo scala (saturazione) a circa 3-5 cm dalla lampadina, mantenendo però un segnale netto a grandi distanze.* ***Regolata la tensione, non va più toccata né spenta la lampadina.***
- ***Luce Ambientale (Segnale Netto):*** *Il segnale reale va calcolato come $V_i = V_{R,i} - V_{F,i}$. Si legge prima la tensione a luce diretta (V_R), poi si inserisce*

*rapidamente un oscuratore ("monetina") per schermare solo il fascio diretto e si legge il fondo (V_F). **Non cambiare mai scala sul voltmetro** durante questa coppia di letture. La monetina dovrebbe essere inserita inizialmente a metà tra fotodiodo e lampadina; successivamente, spostando il fotodiodo, si dovranno provare diverse posizioni annotando la tensione di fondo in queste ultime (es. 3 misure di fondo) per poi farne una media. Per calcolare l'errore si dovrà fare la semidifferenza tra il valore massimo e il minimo divisa per $\sqrt{3}$; se le misure di fondo avranno sempre lo stesso valore, l'errore corrisponderà alla sensibilità (precisione) del voltmetro.*

- ***Campionamento Disomogeneo:** A brevi distanze (3-15 cm) si prendono molti punti ravvicinati (passi da 2-5 mm) per campionare l'alta pendenza; a medie e grandi distanze si allarga il passo a 5-10 cm verso fine binario.*

1.4.3 Analisi Dati e Preparazione per ROOT

ATTENZIONE agli Errori nel Fit: Per eseguire il fit (es. in ROOT), si assegna ai punti **solo** l'incertezza sulla coordinata x_i (ovvero σ_x) e l'errore di lettura del voltmetro su V_i. **NON propagare subito** x_0 **e** D nei punti sperimentali, altrimenti si introdurrebbero false correlazioni che sballerebbero i risultati.

La funzione di **Fit 1** dovrà avere una forma del tipo:

$$f(x) = \frac{A}{(x-B)^2} + C \tag{1.42}$$

con A, B, C parametri da stimare. In particolare: $A \neq 0$. Il parametro B stima lo shift spaziale residuo: a fit concluso, per verificare che sia compatibile con 0, dovremo confrontarlo **considerando finalmente gli errori comuni** σ_{x_0} e $\sigma_{D/2}$. Anche l'offset di tensione C deve risultare compatibile con 0 ($C \pm \sigma_C \simeq 0$).

Se le misure saranno state prese abbastanza bene, l'errore su C dovrebbe essere confrontabile con gli errori della tensione ($\sigma_V = \sqrt{\sigma_{lett}^2 + \sigma_{fondo}^2}$, anche se in realtà dovrebbe essere un po' più piccolo) e il valore $C \pm 2\sigma_C$ dovrebbe risultare compatibile con 0. Anche il parametro B dovrebbe essere compatibile con 0, ma in questo caso la compatibilità andrà verificata tenendo conto della stima $B \pm \sigma_B \pm \sigma_{x_0} \pm \sigma_{D/2} \pm \sigma_{1\text{mm}}$. Il parametro A, invece, rappresenta un fattore di scala che ci indica l'ampiezza (altezza) della curva da determinare.

Come prova ulteriore della dipendenza $1/r^2$, è richiesto eseguire un **Fit 2**, usando come parametri iniziali A, B, C quelli ottenuti nel primo, e aggiungendo il parametro esponente D:

$$f_2(x) = \frac{A}{(x-B)^D} + C \tag{1.43}$$

verificando che l'esponente stimato sia compatibile con 2.

1.4.4 Grafici Diagnostici Finali

Per un'analisi completa e corretta, la relazione di laboratorio dovrà presentare **tre grafici**:

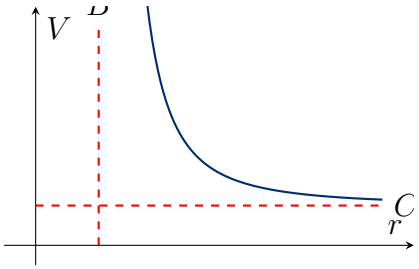
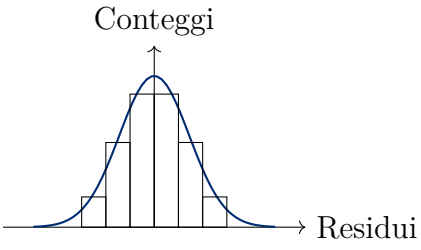
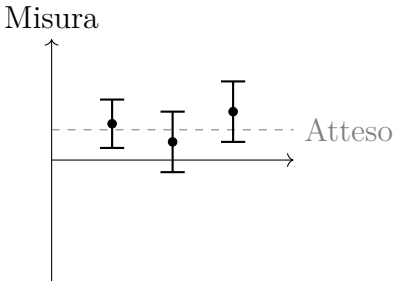


Figura 1.5: Curva di tensione e Asintoti.



(a) Istogramma dei Residui.



(b) Compatibilità Misure.

Figura 1.6: Diagnostica statistica post-fit.